ESSILORLUXOTTICA COLOMBIA · DOCUMENTO NORMATIVO INTERNO

Estándar de integración — DWH corporativo y SAMII

Reglas para construir soluciones de datos dentro de la infraestructura de IT de EssilorLuxottica Colombia. Aplica a **cualquier área** — EHS, Supply Chain, Logística, Calidad, Costos, Nivel de Servicio, Producción — y a **cualquier trabajo**, tanto uno que arranca de cero como uno ya construido que se quiera corregir.

---

0. Cómo se lee este documento

Cada afirmación de aquí es de uno de tres tipos, y no se mezclan:

TIPO	QUÉ SIGNIFICA	DÓNDE VIVE
Hecho	Verificado contra fuente primaria, con la fuente citada	Secciones 1 a 6
Regla	Obligación derivada de un hecho o de una definición de área	Secciones 2 a 6, numeradas `R.x.y`
Abierto	Sin respuesta. No se decide por defecto ni se asume la opción razonable	Sección 8

Tres cosas que este documento no hace:

1. **No emite veredictos sobre lo ya construido.** Lo que existe hoy puede estar bien o mal; eso se determina aplicándole el checklist de la sección 7, no leyendo aquí una opinión. Ningún trabajo previo se trata como referencia de verdad por el solo hecho de existir.
2. **No convierte una pregunta abierta en una recomendación.** Si algo está en la sección 8 es porque no se sabe. Una respuesta plausible sigue siendo una invención.
3. **No sustituye las definiciones de negocio.** `ehs\CLAUDE.md` manda sobre los cálculos de EHS; `supply-chain\CLAUDE.md` sobre los de demanda. Este documento manda sobre la forma, no sobre el número.

Fuentes primarias usadas. Los dos diagramas de arquitectura que IT compartió (`ehs\diagrama de flujo dataware house.png` y `ehs\data warehouse de estructura de samii(plataforma de descarga de reportes).png`); el hilo de correo del 3 de julio al 26 de agosto de 2026 entre Supply Chain e IT; la transcripción de la sesión con Aquilino Tafur del 22 de julio de 2026; y el código y la documentación en disco de los trabajos existentes. Fecha de corte: **30 de agosto de 2026**.

---

1. Rol

Samuel Galeano es el **implementador**. El área plantea el problema y aporta el conocimiento operativo; el implementador construye el proceso completo: ingesta, modelo, transformación, visualización y migración. El área no escribe código; el implementador no decide reglas de negocio.

R.1.1 — Ningún trabajo pasa a visualización sin haber verificado un período completo contra la fuente original, número por número, con visto bueno del área.

R.1.2 — Lo que se construye para un área tiene que servir para la siguiente. Una solución que solo funciona porque tiene el nombre del área incrustado en el código no cumple.

R.1.3 — Antes de construir, se comprende el problema por escrito: qué decisión se quiere tomar mejor, quién la toma, cada cuánto, con qué información. Sin ese documento no se abre un editor.

---

2. La arquitectura de IT

2.1 El patrón de tres saltos

(Hecho — leído de los dos diagramas de arquitectura.)

IT no conecta nada directamente contra un sistema fuente. El camino es siempre:

```
sistema fuente → JOB → réplica (R-<sistema>) → DWH / SAMII → consumidores
```

Cada sistema operativo tiene una **base espejo** dentro de una instancia de SQL Server controlada por IT, alimentada por un JOB programado. El DWH y SAMII leen de las réplicas, nunca del sistema vivo.

2.2 Instancias

(Hecho — leído de los dos diagramas.)

INSTANCIA	CONTIENE
Instancia DWH BR	`DATAWARE HAUSE` + réplicas `R-VISTAR`, `R-SYMAX`, `R-DMS`, `R-ADATEC`
Instancia SAMII	`SAMII`, `PRUEBAS SAMII` + las mismas cuatro réplicas
Instancia SYMAX BR	`SYMAX`, `SYMAX PRUEBAS`

Existen ambientes de pruebas nombrados: `PRUEBAS SAMII` y `SYMAX PRUEBAS`.

2.3 Sistemas fuente y consumidores

(Hecho — leído de los dos diagramas.)

Fuentes que alimentan las réplicas: `VISTAR` en Celta Park, `VISTAR` en La Estrella (instancias separadas), `SYMAX`, `DMS` (nube), `ADATEC` (nube), y archivos Excel.

Consumidores del DWH: `SAMII`, `SAP BO FR`, archivos Excel y usuarios finales.

SAMII aparece en dos papeles: **lee** del DWH y de las réplicas, y **entrega** a los usuarios. En el diagrama de SAMII, el `DATAWARE HAUSE` llega a la instancia de SAMII por JOB, como una fuente más.

> **Abierto.** En las transcripciones de las áreas aparecen los nombres «CIMAX», «DATEK» y «Hadapec», que podrían corresponder a `SYMAX`, `ADATEC` y otro sistema. No hay ninguna fuente que confirme la correspondencia. No se asume. → Sección 8.1.

2.4 El patrón de base intermedia

(Hecho — transcripción con Aquilino Tafur, 22:16–24:36.)

Cuando un proveedor no permite acceso directo a su base, IT monta una **base intermedia que jala la información en bucle cada 15–20 minutos**. Ya está en producción para la aplicación de mensajeros.

R.2.4.1 — Ante una fuente que no permite conexión directa, se propone este patrón antes que cualquier alternativa: ya existe en la casa y IT lo sabe operar.

---

3. Stack tecnológico

(Hecho — correo de Santiago Murcia del 27 de julio de 2026.)

CAPA	TECNOLOGÍA
Base de datos	**SQL Server**
Backend	**Node.js v22.19.0** (versión exacta)
Plantillas	**EJS** sobre HTML semántico
Estilos	**CSS** — Flexbox, Grid, variables
Cliente	**JavaScript** — DOM, fetch, promesas, asincronía
Gráficas	**Chart.js** o **ECharts**
Versionado	**Git** + **GitHub** con cuenta corporativa
Editor	**VS Code** + ESLint, Prettier, Error Lens, Code Runner, Console Ninja, Live Server, Rest Client
Cliente SQL	**SQL Server Management Studio**
Ruta de proyectos	`c:/proyectos/essilor`
IA de desarrollo	**Codex** — IT ofrece indicar la forma de uso

R.3.1 — Los dashboards que van a SAMII se construyen con Chart.js o ECharts. No se introduce otra librería de gráficas.

R.3.2 — Node se fija en v22.19.0. No se sube la versión por conveniencia local: el proyecto es compartido.

R.3.3 — ESLint y Prettier no son opcionales. Son lo que evita que la conciliación de la rama genere ruido de formato.

R.3.4 — No se introduce ninguna dependencia que no esté en esta tabla sin acordarlo con IT primero.

3.5 Los módulos de SAMII

(Hecho — correo de Santiago Murcia del 21 de julio de 2026.)

MÓDULO	FUNCIÓN
Dashboards	Visualización analítica y detallada
Reportes	Consultas dinámicas sobre el DWH o sobre datos propios de SAMII
Formularios	Captura con trazabilidad de un proceso específico
Maestros	Mantenimiento de catálogos que no viven en otros sistemas
Procesos	Desarrollos puntuales que no encajan en otro módulo
Admin	Roles, permisos, usuarios, acceso a módulos

R.3.5.1 — Antes de proponer algo nuevo se comprueba si cabe en un módulo existente. La propuesta de IT es literal: *«cada proceso nuevo que quepa en los módulos actuales lo podemos integrar debidamente»*.

R.3.5.2 — El módulo Admin no se toca. Lo administra Jorge Salcedo.

---

4. Flujo de trabajo

(Hecho — correo de Santiago Murcia del 21 de julio de 2026, ocho pasos con responsables.)

#	PASO	RESPONSABLE
1	Definir el alcance del proceso en la rama de desarrollo propia	Implementador
2	Estandarización del proceso desarrollado	Implementador + Santiago
3	Fase de pruebas	Implementador
4	Conciliación hacia la rama maestra de desarrollo	**Santiago**
5	Normalización en la rama de integración maestra	**Santiago**
6	Asignación de permisos	Santiago + implementador · administra Jorge Salcedo
7	Paso a producción	**Santiago**
8	Validación de funcionalidad en producción	Implementador

R.4.1 — El implementador desarrolla y prueba en su rama. **No concilia hacia la maestra ni despliega a producción.**

R.4.2 — Todo desarrollo pasa por fase de pruebas antes del paso 4. No hay ruta rápida.

R.4.3 — El SQL se versiona como archivos en el repositorio, no solo dentro de la base. El paso a producción es ejecutar scripts revisados.

R.4.4 — Commits pequeños, con mensaje que describa el cambio. `git pull` antes de empezar la jornada.

5. Reglas de construcción

5.1 Nomenclatura

R.5.1.1 — `snake_case` en tablas, columnas, vistas y procedimientos. Es la convención de IT.

R.5.1.2 — Sin tildes ni ñ en identificadores de código, nombres de archivo ni de carpeta. En el texto de los documentos, con normalidad.

R.5.1.3 — Cuando un modelo existente usa otra convención, la traducción ocurre **en la frontera**: lo que entra a SQL Server va en snake_case. No se crean objetos nuevos con la convención vieja.

5.2 Dónde vive el cálculo

R.5.2.1 — Los indicadores se calculan **en la base de datos**, en la vista o el procedimiento que alimenta el consumidor. No en el JavaScript del front, no en el DAX de un informe, no en un script suelto.

R.5.2.2 — Una regla de cálculo tiene **una sola implementación**. Cualquier otra capa la lee; no la reimplementa. Dos implementaciones de la misma regla son dos oportunidades de divergir.

R.5.2.3 — La tabla de hechos guarda **magnitudes crudas** al grano más fino. Los ratios se derivan al consultar. Nunca se guarda un ratio precalculado.

R.5.2.4 — Los parámetros de negocio — metas, umbrales, factores de conversión — viven en **tabla editable**, nunca en código. Cambiar una meta debe ser editar una fila.

R.5.2.5 — El grano de la tabla de hechos se decide antes que nada y se decide fino. Siempre se puede agregar hacia arriba; nunca se puede desagregar lo que se guardó grueso.

R.5.2.6 — El texto analítico también es cálculo. Una frase que afirma algo sobre un número sale de una rama sobre ese valor, no de una redacción.

5.3 Ingesta

R.5.3.1 — Tres puertas, y cada dato usa una sola: 1. **Captura manual** → formulario de SAMII, que escribe a SQL Server con validación y registro de quién y cuándo. No un Excel compartido. 2. **SharePoint** → extractor programado hacia staging. 3. **Sistemas operativos** → la réplica `R-*` correspondiente de IT.

R.5.3.2 — El extractor **no interpreta**. Trae crudo a staging; la limpieza ocurre después, en SQL, donde queda versionada y auditable.

R.5.3.3 — Toda carga es **idempotente**: correrla dos veces no duplica datos.

R.5.3.4 — Toda carga escribe en **bitácora**: cuándo corrió, cuántas filas movió, si falló. Sin esto, un cero en un tablero no se puede diagnosticar.

R.5.3.5 — Lo que no pasa validación **no entra**: se aparta en tabla de rechazos con su motivo. Nunca se corrige a mano en el destino.

R.5.3.6 — **Una celda vacía no es un cero.** El modelo y el consumidor distinguen «sin dato» de «valor cero».

R.5.3.7 — Una fuente que depende de que una persona actualice un archivo es una fuente de primera clase, no un defecto a ignorar. Se migra a formulario; no se elimina ni se suple con un supuesto.

5.4 Datos y seguridad

R.5.4.1 — Los datos crudos no se versionan. `.xlsx`, `.txt`, exports y volcados van fuera de git.

R.5.4.2 — **El dato real no sale del tenant corporativo.** Publicar información operativa en un servicio externo no está permitido.

R.5.4.3 — En personas naturales el NIT **es la cédula**. Cualquier tabla que lo contenga es tratamiento de datos personales bajo la **Ley 1581 de 2012**: no se expone en API, no se versiona, y se seudonimiza en las vistas de consumo.

R.5.4.4 — Antes de cualquier comando `vercel` que cree, modifique o borre algo, se ejecuta `vercel whoami` y `vercel teams ls`, se muestra el resultado y se espera confirmación explícita. *(Heredada de `CLAUDE.md` raíz, sección 6.)*

R.5.4.5 — Un hallazgo de exposición de datos personales se escala por su canal — jefatura, seguridad de la información, jurídica — y no se queda como línea en un documento técnico.

5.5 Verificación

R.5.5.1 — Nada se afirma sin verificar. Un número faltante es mejor que un número inventado.

R.5.5.2 — Toda migración tiene **compuerta**: un verificador que compara origen y destino y sale con error si hay diferencia. No se cambia el motor de lectura hasta que la compuerta pase sin diferencias.

R.5.5.3 — Una cifra vive en **un solo lugar**. Copiarla a un README, a un CLAUDE.md y a una documentación técnica garantiza que se desincronicen.

R.5.5.4 — Un guardarraíl se prueba **en negativo**: se inyecta el error que debe detectar y se comprueba que falla. Un validador que nunca falló no está probado.

---

6. Migración de histórico heredado

Aplica a cualquier área que traiga años de datos en Access, Excel o exports planos.

6.1 Lo que enseña el caso de Supply Chain

(Hecho — `supply-chain\demanda-historica\docs\recorrido-del-dato.md` y transcripción con Aquilino Tafur.)

El histórico de demanda llegó como **nueve archivos de texto plano, uno por año 2018–2026**, exportados a mano desde una base **Microsoft Access** que Aquilino Tafur mantiene con VBA.

HECHO	DETALLE
Por qué están separadas	Access topa cerca de **2 GB**; se genera una base nueva cada año por peso
Volumen	**17.457.959 líneas**, 3.662.550.782 bytes
Esquemas	**Tres distintos**: 15 columnas (2018, 2019, 2024, 2025), 14 (2020–2023), 22 (2026)
Encoding	Mixto: cuerpo UTF-8 con ~3,3 millones de bytes sueltos cp1252; cabeceras cp1252
Desalineación	`2022.txt` contiene **306.662 filas que son de 2023**
Datos personales	El NIT es cédula en personas naturales

La causa raíz: **nadie definió un contrato de exportación**, así que cada año salió con las columnas que en ese momento parecieron útiles. No hay claves foráneas, ni catálogos, ni restricciones.

Estas nueve fotografías no son nueve bases. Son nueve capturas planas de la misma base, tomadas en momentos distintos y con encuadres distintos. Unificar significa reconstruir el modelo que esas exportaciones perdieron.

6.2 El Access vivo es otra cosa

(Hecho — transcripción con Aquilino Tafur, 18:00–53:33.)

Además del histórico exportado, hay bases Access **en producción diaria**: locales en el equipo, con las tablas vinculadas a SharePoint, respaldo por OneDrive, carga por asistentes de importación, consultas SQL y automatización en VBA. Tres fuentes se unen ahí, y una de ellas se llena a mano día a día.

El cuello de botella medido **no es el procesamiento** (5–10 segundos) sino la extracción y la dependencia de que la persona actualice su archivo.

R.6.2.1 — Un histórico exportado y una base viva son dos trabajos distintos con dos soluciones distintas. No se mezclan en el mismo plan de migración.

6.3 Reglas de migración

R.6.3.1 — **El año de una fila se toma del DATO, nunca de la RUTA.** Un archivo llamado `2022.txt` no garantiza que su contenido sea de 2022. En Parquet particionado, nunca leer con `hive_partitioning=true`: la columna del directorio enmascara la del archivo.

R.6.3.2 — El superconjunto de columnas se resuelve **por nombre, nunca por posición**. Con varios esquemas, la posición miente.

R.6.3.3 — Prohibido `errors='ignore'`, `errors='replace'` o `iconv -c` sobre el crudo. Descartar bytes es perder dato en silencio.

R.6.3.4 — Antes de comparar dos períodos se verifica que sean comparables. Un histórico largo casi nunca es homogéneo: cambian esquemas, cambian nombres de sitio, cambia el grano.

R.6.3.5 — El grano común a todos los años manda, aunque el año más reciente traiga más detalle. No se modela una jerarquía que ocho de nueve años no pueden llenar.

R.6.3.6 — Antes de dar por buena una migración se busca un **total en otro sistema** para reconciliar. Sin control externo, la verificación solo comprueba que el proceso es consistente consigo mismo.

R.6.3.7 — Toda migración de histórico deja por escrito las **preguntas al origen**: qué campo define la fecha, por qué desapareció una columna, qué significa una categoría que aparece un solo año. Esas preguntas van al dueño del dato, no se resuelven por inferencia.

---

7. Cómo se aplica esto a un trabajo

7.1 Trabajo nuevo

1. **Comprender el problema** con el área, por escrito (R.1.3).
2. **Inventariar las fuentes**: dónde vive hoy cada dato, qué puerta de ingesta le corresponde (R.5.3.1), quién es su dueño, con qué frecuencia cambia.
3. **Diseñar el modelo**: grano, hechos, dimensiones (R.5.2.3, R.5.2.5). Validarlo contra las preguntas del paso 1 y revisarlo con IT antes de crear nada — puede que parte ya exista en el DWH.
4. **Construir la ingesta**: staging, validación en frontera, cargas idempotentes, bitácora (R.5.3.2 a R.5.3.6).
5. **Verificar contra la realidad**: un período completo cuadrado contra el origen, con visto bueno del área (R.1.1).

6. **Construir la visualización** con el stack de la sección 3, cálculo en la base (R.5.2.1).
7. **Pasar por el flujo de IT** (sección 4).
8. **Automatizar la vigilancia** — centinelas de calidad — antes de añadir análisis.

7.2 Trabajo en curso que se quiere modificar

No se juzga por lo que se le invirtió ni por lo bien resuelto que esté internamente. Se le pasa este checklist y se anota qué regla incumple. Lo que incumple se corrige o se sustituye; qué de las dos cosas es una decisión, no un resultado del checklist.

#	COMPROBACIÓN	REGLA
1	¿El cálculo vive en la base de datos, y en un solo lugar?	R.5.2.1, R.5.2.2
2	¿La tabla de hechos guarda magnitudes crudas, no ratios?	R.5.2.3
3	¿Metas y parámetros están en tabla editable, no en código?	R.5.2.4
4	¿La nomenclatura es snake_case, sin tildes ni ñ?	R.5.1.1, R.5.1.2
5	¿El stack coincide con la sección 3? ¿Gráficas en Chart.js o ECharts?	R.3.1, R.3.4
6	¿Cada fuente entra por una sola de las tres puertas?	R.5.3.1
7	¿La carga es idempotente y deja bitácora?	R.5.3.3, R.5.3.4
8	¿Se distingue «sin dato» de «cero»?	R.5.3.6
9	¿El dato real está dentro del tenant?	R.5.4.2
10	¿Hay datos personales expuestos en API, en git o sin seudonimizar?	R.5.4.3
11	¿Cada cifra publicada vive en un solo lugar?	R.5.5.3
12	¿La migración, si la hay, tiene compuerta que pase sin diferencias?	R.5.5.2
13	¿Los guardarraíles están probados en negativo?	R.5.5.4
14	¿El SQL está versionado como archivos?	R.4.3
15	¿La solución sirve para otra área, o tiene el área incrustada?	R.1.2

R.7.2.1 — El checklist se aplica **antes** de tocar código, y su resultado se escribe. Corregir sin haber listado qué incumple produce arreglos parciales que se descubren tarde.

R.7.2.2 — Cuando la corrección implica ejecutar algo en el servidor de IT, ese paso va **primero** y con su compuerta en verde (R.5.5.2). Retirar una implementación antes de que su reemplazo esté verificado deja el trabajo sin ninguna.

7.3 Piloto

R.7.3.1 — El primer trabajo dentro del entorno de IT es **pequeño y completo**: un solo proceso, de la fuente al consumidor en producción, recorriendo los ocho pasos del flujo. Sirve para descubrir el entorno real y para probar que el flujo con IT funciona.

R.7.3.2 — Un piloto no es un prototipo. Termina en producción con dato real y usuarios reales, o no fue piloto.

R.7.3.3 — Lo que el piloto descubra sobre el entorno se escribe como regla en este documento antes de arrancar el segundo trabajo.

---

8. Lo que no está definido

Ninguna de estas preguntas tiene respuesta. No se decide por defecto ni se asume la opción más razonable. Se preguntan, y la respuesta se escribe como regla en la sección que corresponda.

8.1 El DWH

- ¿Cómo se organizan las capas internas y qué convención de nombres siguen?
- ¿Qué fuentes están conectadas hoy y con qué frecuencia corren los JOB?
- ¿Existen dimensión fecha y dimensión sitio/centro corporativas que deba reutilizar en vez de crear las mías?
- ¿Con qué se orquestan los ETL — SQL Server Agent u otra herramienta? ¿Hay bitácora de cargas estándar?
- ¿Qué datos de laboratorio ya viven en el DWH: producción, horas-hombre, consumos?
- ¿Cómo trata el DWH los datos personales? ¿Hay política de seudonimización?
- ¿A qué sistemas corresponden los nombres «CIMAX», «DATEK» y «Hadapec» de las transcripciones?

8.2 Mi espacio de trabajo

- ¿Esquema propio por área con permisos DDL y DML? ¿En qué servidor de desarrollo?
- ¿Cómo es el paso de objetos de base de datos de desarrollo a producción?
- ¿La rama es sobre el repositorio completo de SAMII o hay repos separados por módulo?
- ¿Cómo se levanta SAMII en local? ¿Hay guía, base de desarrollo, datos de prueba? ¿Es `PRUEBAS SAMII` ese ambiente?

8.3 Ingesta

- ¿Cómo resuelve IT hoy la extracción desde SharePoint — Graph API, Power Automate, lectura programada?
- El módulo de Formularios, ¿se configura o se programa? ¿Quién define las validaciones?
- ¿Un área puede seguir capturando en su Excel de SharePoint, o el dato debe entrar por formulario de SAMII?

8.4 Histórico heredado

- ¿Dónde aterriza un histórico migrado: esquema dentro del DWH, base propia en esa instancia, o dentro de SAMII?
- ¿El patrón de réplica `R-*` aplica también a una base Access de área, o el histórico se carga una vez y se corta el origen?
- ¿Se puede fijar un **contrato de exportación** — columnas, grano, formato, criterio de corte anual — con el dueño del dato?
- ¿Se pueden re-exportar los años viejos con el esquema completo del año más reciente?
- ¿La migración la ejecuta el implementador con credenciales, o se entregan los scripts a IT?

8.5 Visualización

- ¿Chart.js o ECharts, o depende del caso?
- ¿Hay componentes o plantillas de dashboard ya hechos en SAMII para reutilizar?
- ¿Qué pasa con lo ya construido fuera del stack — se porta, se migra o se rehace?

- ¿Power BI y SAMII conviven, o SAMII es el destino de los tableros nuevos?
- ¿Licencia Power BI y workspace: existen y quién los asigna?

8.6 Método

- ¿Cuáles son los lineamientos de la casa para desarrollar con IA (Codex)?
- ¿Hay ambiente donde probar una migración completa antes de tocar la base real?
- ¿Cuál sería el primer trabajo apropiado como piloto (R.7.3.1)?

---

9. Estructura de carpetas

```
<area>\
├── CLAUDE.md            definiciones de negocio — mandan sobre el cálculo
└── <trabajo>\
    ├── CLAUDE.md · CONTEXTO.md
    ├── datos\           insumos crudos — nunca se versionan
    ├── analisis\        exploración y validación
    ├── docs\            especificación, decisiones, bitácora
    └── app\             el repositorio git, si el trabajo produce software
```

R.9.1 — Por cada área, un documento vivo con: las preguntas de negocio, el inventario de fuentes, el modelo, las reglas de cálculo de cada indicador y las decisiones con su porqué.

R.9.2 — Repositorio limpio: lo que llega a un remoto es exactamente `app\`, sin rastro de herramientas de IA. Se consigue con `.git\info\exclude`, que es local, **no** con `.gitignore` — una línea con `.claude/` en el `.gitignore` ya sería un rastro visible.

R.9.3 — Una carpeta documentada como capa activa existe y tiene contenido, o no se documenta.

Apéndice — Trazabilidad de los hechos

SECCIÓN	FUENTE PRIMARIA
2.1, 2.2, 2.3	`ehs\diagrama de flujo dataware house.png` · `ehs\data warehouse de estructura de samii(plataforma de descarga de reportes).png`
2.4, 6.1, 6.2	Transcripción «Proceso de supply entender la parte visual y estructura de Access con Aquilino», 22 de julio de 2026
3, 3.5, 4	Correo «RE: Solicitud de creación de base de datos en ambiente de desarrollo», Santiago Murcia, 21 y 27 de julio de 2026
5.4.4, 9.2	`EssilorLuxottica\CLAUDE.md`, secciones 2 y 6
6.1, 6.3	`supply-chain\demanda-historica\docs\recorrido-del-dato.md` · `supply-chain\CLAUDE.md`
5.2, 5.3.6	`ehs\CLAUDE.md` — definiciones y auditoría del Excel heredado